

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1: Experiments

**COURSE NAME:** Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

---

### COURSE OUTCOME COVERED

**CO3:** *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications.* (BL3)

Assessment Tool Weightage: 40%

Dave's Taxonomy: Precision (Level 3)  
Question Count: 5

**Total Marks: 40**

---

### Code of Conduct

I **Y.Sai Mohith(192372210)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Y.Sai Mohith

### Assessment Tasks

#### Experiments

### Experiment 1

**Question:** Create and document a cloud storage comparison between object, block, and file storage. Select the best storage model for backups, VM disks, and shared enterprise documents.

**Objective:** To compare object, block, and file storage and select the best storage model for backups, VM disks, and shared enterprise documents.

### Architecture Sketch:

Architecture Sketch



**Procedure:** 1. Identify the storage requirements of each workload.

2. Compare object, block, and file storage based on access method, performance, scalability, and sharing.
3. Select object storage for backups because it is suitable for large unstructured data.
4. Select block storage for VM disks because virtual machines require block-level access.
5. Select file storage for shared enterprise documents because multiple users can access a common file system.
6. Record the comparison and justify each selection.

**Expected Output:** Object storage is selected for backups, block storage for VM disks, and file storage for shared enterprise documents. The comparison clearly maps each workload to its suitable storage model.

**Conclusion:** Each storage model supports a different workload. Correct selection improves performance, scalability, accessibility, and cost efficiency.

## Experiment 2

**Question:** Develop a simple REST API workflow for a cloud-based student portal. Show request, response, authentication, and deployment flow.

**Objective:** To develop a simple REST API workflow for a cloud-based student portal showing request, response, authentication, and deployment flow.

### Architecture Sketch:

Architecture Sketch



**Procedure:**

1. Define REST endpoints such as student, course, and attendance services.
2. The client sends an HTTPS request to the required endpoint.
3. Authenticate the user using credentials or a token.
4. Validate the request and process it in the cloud application.
5. Read or update the required data.
6. Return a JSON response with a suitable status code.
7. Deploy the API on a cloud platform and test the complete workflow.

**Expected Output:** The authenticated client sends a REST request to the deployed student portal and receives an appropriate JSON response.

**Conclusion:** REST APIs provide a standard communication method between clients and cloud applications. HTTPS and authentication improve security, while cloud deployment supports accessibility and scalability.

## Experiment 3

**Question:** Design a serverless event flow for image upload processing using object storage, function-as-a-service, and notification service.

**Objective:** To design a serverless event flow for automatic image upload processing using object storage, function-as-a-service, and a notification service.

### Architecture Sketch:

Architecture Sketch



- Procedure:**
1. Create an object storage location for uploaded images.
  2. Configure an upload event as the trigger.
  3. Connect the event to a serverless function.
  4. The function reads and processes the uploaded image.
  5. Store the processed image in the required location.
  6. Trigger the notification service after successful processing.
  7. Test the workflow using sample images and monitor function execution.

**Expected Output:** An uploaded image automatically triggers the serverless function, processes the image, stores the result, and generates a notification.

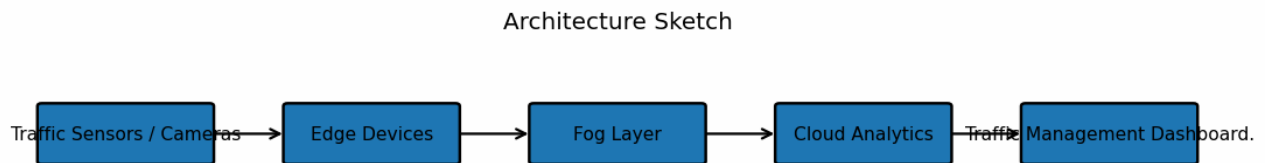
**Conclusion:** The serverless event flow provides automatic and scalable image processing without direct server management. Object storage, functions, and notification services work together efficiently.

## Experiment 4

**Question:** Compare edge computing and fog computing for a smart traffic management system. Include architecture, latency considerations, and deployment location.

**Objective:** To compare edge and fog computing for a smart traffic management system, including architecture, latency, and deployment location.

### Architecture Sketch:



**Procedure:**

1. Collect traffic information from sensors and cameras.
2. Process urgent information close to the sensors using edge computing.
3. Use nearby fog nodes for coordinated local processing.
4. Send selected or summarized information to the cloud for analytics and storage.
5. Compare latency, bandwidth usage, processing location, and response time.
6. Use edge processing for immediate traffic-control decisions and fog processing for coordinated local analysis.

**Expected Output:** Edge computing provides very low latency for immediate traffic decisions, while fog computing provides an intermediate distributed processing layer between edge devices and the cloud.

**Conclusion:** Edge computing is suitable for immediate local decisions, while fog computing is useful when nearby devices require coordinated processing. Both reduce unnecessary dependence on centralized cloud processing.

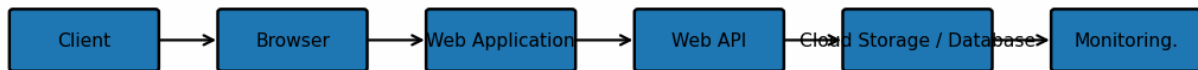
## Experiment 5

**Question:** Demonstrate a cloud application deployment workflow showing client access, browser interaction, web API call, storage access, and monitoring.

**Objective:** To demonstrate a complete cloud application deployment workflow covering client access, browser interaction, web API calls, storage access, and monitoring.

### Architecture Sketch:

Architecture Sketch



- Procedure:**
1. Deploy the web application on a cloud platform.
  2. Configure the application URL and access settings.
  3. Open the application using a browser.
  4. Perform a user action that generates a web API request.
  5. Authenticate and process the request.
  6. Access or update data in cloud storage.
  7. Return the result to the browser.
  8. Enable monitoring for availability, response time, errors, and resource usage.
  9. Test the complete deployment workflow.

**Expected Output:** The client accesses the browser application, the application calls a web API, the API accesses cloud storage, and monitoring records application activity.

**Conclusion:** Combining client access, browser interaction, web APIs, cloud storage, and monitoring provides a practical and manageable cloud deployment workflow.